

¡Entendido! Al ser una aplicación construida con **Electron (HTML/CSS/JS)**, tienes una ventaja enorme: puedes aprovechar todas las librerías modernas de visualización web para que esa interfaz no solo se vea bien, sino que sea "viva".

Para diferenciarlo de un GPS común y hacerlo una verdadera herramienta de **Simulación**, el enfoque debe ser la **interactividad y la exposición de datos**.

Aquí tienes cómo elevar el nivel de tu proyecto con tecnologías web puras:

1. Haz que el mapa sea "Inteligente" y no Estático

Un GPS solo te guía. Una simulación te permite **experimentar**.

- **Añade Obstáculos Manuales:** Usa eventos de clic en el mapa para que, si el usuario hace clic en una calle, esta se "bloquee" (puedes pintar una X roja con CSS o Canvas). Al instante, dispara tu lógica de JS para recalculando la ruta óptima. Esto demuestra que tu software **reacciona a cambios en el modelo**.
- **Capas de Calor (Heatmaps):** Usa librerías como Leaflet.heat (si usas Leaflet) para mostrar zonas de alta congestión de pedidos. Esto visualmente dice: "Aquí es donde más se necesita optimizar".

2. Implementa un "Dashboard de Telemetría"

En lugar de solo mostrar números estáticos abajo, crea un panel lateral que use **gráficos en tiempo real**.

- **Gráficos de Eficiencia:** Usa [Chart.js](#) para mostrar una comparativa de: *Tiempo Estimado Original* vs. *Tiempo con Optimización*. Ver las barras moverse cuando cambia la ruta es muy impactante para un jurado.
- **Estado de los Agentes:** Si simulas varios vehículos, crea "tarjetas" dinámicas en el HTML que muestren el estado de salud de la misión (Batería/Gasolina, cantidad de paquetes, siguiente parada).

3. El "Modo Simulación" (Play/Pause/Rewind)

Un GPS siempre está en "tiempo real". Una simulación debe poder controlarse.

- **Controles de tiempo:** Agrega botones de Play, Pause y un slider de Velocidad de Simulación (x1, x2, x4).
- **Log de Eventos:** Crea una consola pequeña (un <div> con scroll) donde JS imprima lo que el "cerebro" del software está haciendo:
 - [14:20] Analizando 450 rutas posibles...
 - [14:21] Ruta óptima encontrada vía Av. Jacinto Lara.
 - [14:22] Alerta: Tráfico detectado en Punto Fijo, recalculando...

4. Mejoras Estéticas y de Intuitividad (UI/UX)

Para que sea más intuitivo sin usar frameworks pesados:

- **Glassmorphism:** Ya tienes un buen fondo oscuro. Usa backdrop-filter: blur(10px); en CSS para tus paneles flotantes (como el de "Control de Despacho"). Esto hace que la interfaz se sienta moderna y "Premium".
- **Micro-interacciones:** Cuando el software encuentre una ruta mejor, haz que la línea en el mapa brille o cambie de color brevemente. Usa CSS Animations para las transiciones

de los paneles.

- **Tooltips Informativos:** Si el usuario pasa el mouse sobre un punto de la ruta, muestra un pequeño cuadro con el "Costo de Peso" (el valor que el algoritmo le asignó a ese tramo).

Herramientas recomendadas para tu Stack (JS puro):

- **Librería de Mapas:** Si no la usas ya, **Leaflet.js** es mucho más ligera y personalizable para Electron que Google Maps.
- **Iconos:** **FontAwesome** o **Lucide Icons** para que los botones de navegación y configuración sean universalmente entendibles.
- **Notificaciones:** **SweetAlert2** para avisos importantes cuando el sistema termine de calcular una ruta compleja.

El diferenciador para tu profesor:

Dile que no es un GPS porque el GPS es **reactivo** (te dice por donde ir), mientras que tu software es **proactivo y analítico**: permite simular fallos, cambiar algoritmos en caliente y visualizar métricas de rendimiento que un GPS comercial nunca te daría.

¿Qué algoritmo de búsqueda de rutas estás implementando actualmente en tu JavaScript?